CVE-2026-42897 Defensive Lab and Mitigation Validation Report

Prepared for: Josh Parris

Date: 5 June 2026 (Australia/Sydney)

Table of Contents

Document purpose and safety boundary

This report is a defensive, IT-manager-oriented lab guide for CVE-2026-42897 in Microsoft Exchange Outlook Web Access (OWA). It is designed to help a manager or administrator understand the risk, validate Microsoft mitigations, improve logging, and make a defensible production decision.

This report does not reproduce the live exploit. It does not include the exact malicious email structure, OWA sanitisation bypass details, session theft instructions, MFA bypass steps, or any payload designed to execute JavaScript inside OWA. Where a requested test would require Exchange-specific exploit material, the report replaces it with safe defensive validation.

The core operational position is:

> Treat CVE-2026-42897 as actively exploited and mitigated, not permanently patched, until Microsoft publishes a security update or hotfix build that supersedes the mitigation for your Exchange version.

1. Executive summary

1.1 What CVE-2026-42897 is

CVE-2026-42897 is a Microsoft Exchange Server vulnerability described by NVD, using Microsoft's CNA data, as improper neutralization of input during web page generation, also known as cross-site scripting (CWE-79). Microsoft rates the issue CVSS 3.1 8.1 High with network attack vector, no privileges required, user interaction required, high confidentiality impact, high integrity impact, and no availability impact. NVD also lists the CVE in CISA's Known Exploited Vulnerabilities (KEV) catalogue. [S1]

Microsoft's Exchange Team disclosed on 14 May 2026 that CVE-2026-42897 affects Exchange Outlook Web Access (OWA). The public attack condition is that an attacker can send a specially crafted email to a user; if the user opens the email in OWA and certain interaction conditions are met, arbitrary JavaScript can execute in the browser context. [S2]

1.2 Who is affected

The affected scope is on-premises Microsoft Exchange Server environments serving OWA, including Exchange Server Subscription Edition (SE), Exchange Server 2019, and Exchange Server 2016 as listed in Microsoft's mitigation table. Exchange Online is reported by Microsoft-derived coverage as not affected. Because Exchange 2016 and 2019 reached end of support on 14 October 2025, production remediation planning should account for support status and Extended Security Update eligibility. [S3][S4]

1.3 Patch and mitigation status as of 5 June 2026

No permanent public code-fix build was identified during this research. The strongest evidence is Microsoft published mitigation code in the CSS-Exchange repository. That code considers a code fix applied only when ExSetup.exe has a file version greater than or equal to 15.2.2562.99. Microsoft's public Exchange Server SE build table currently lists the latest SE build as May26HU 15.2.2562.41, which is below that threshold. [S5][S6]

Microsoft's official interim protection is Mitigation M2, implemented through Exchange Emergency Mitigation (EM) service or the Exchange On-Premises Mitigation Tool (EOMT). Microsoft Learn lists M2 as a mitigation for CVE-2026-42897 via URL Rewrite configuration, applicable from Exchange SE RTM, Exchange 2019 RTM, and Exchange 2016 RTM. [S3]

1.4 Why mitigation validation matters

Exchange EM mitigations are temporary interim fixes. Microsoft explicitly says each mitigation is temporary until the Security Update that fixes the vulnerability is applied, and that EM service is not a replacement for Exchange Security Updates. [S3]

For CVE-2026-42897, the mitigation is not a normal "install patch and reboot" state. Administrators need evidence that:

1. The EM service is enabled and can reach Microsoft's Office Config Service (OCS).
2. The M2 mitigation has been applied or EOMT has applied the same mitigation.
3. The expected OWA outbound URL Rewrite rule is enabled.
4. Relevant OWA HTML shell responses contain the expected Content Security Policy addition.
5. OWA still works for users after mitigation.
6. Any suspected pre-mitigation exposure has been investigated.

1.5 Why this lab does not reproduce the live exploit

The exact exploit chain is not publicly and authoritatively documented. Providing a working Exchange-specific payload for an actively exploited, not-yet-permanently-patched OWA vulnerability would create avoidable harm. A safe lab can still teach the important defensive lessons: Exchange build validation, EM/EOMT verification, CSP header verification, functional testing, logging, incident response, and a separate toy demonstration of stored XSS and CSP in a non-Exchange app.

2. Evidence classification

Claim	Classification	Basis	Practical response
CVE-2026-42897 is an XSS/web page generation flaw in Microsoft Exchange Server	Confirmed	NVD/CNA description and CWE-79	Treat as browser-context script-execution risk in OWA
It is known exploited	Confirmed	CISA KEV entry, NVD KEV section	Urgent mitigation and investigation
Microsoft released EM mitigation M2 via URL Rewrite	Confirmed	Microsoft Learn EM mitigation list	Verify M2 status per server
The EOMT rule appends script-src-attr 'none' to OWA HTML responses	Confirmed	Microsoft CSS-Exchange mitigation code	Verify response header and outbound rule
Permanent public patch is available	Not confirmed	No public build identified meeting EOMT code-fix threshold	Record as mitigated, not patched
The issue is definitely stored XSS in the email body	Unconfirmed	Plausible but Microsoft has not publicly named the vulnerable field	Do not build detection around guessed payload strings
The attack steals session tokens or bypasses MFA	Unconfirmed	Often claimed by third parties but not published by Microsoft as confirmed CVE behaviour	Investigate sessions and mailbox activity as precaution, not as proven outcome
Attackers can create forwarding rules	Unconfirmed	Possible after browser-context compromise but not confirmed for this CVE	Hunt mailbox rules as a prudent response
Opening/previewing alone is sufficient	Unconfirmed	Microsoft says opening in OWA plus “certain interaction conditions”	User reports should capture exact user interaction
Public IOCs are available	Not confirmed	No official payload/IOC set identified	Use behavioural hunting and evidence preservation

3. Threat model

3.1 Assets at risk

- On-premises Exchange OWA service.
- Mailboxes accessed through OWA.
- Browser sessions of users who open attacker-controlled messages in OWA.
- Integrity of mailbox-visible actions performed through a user's authenticated browser context.
- Organisational trust in email, because spoofing and mailbox actions can have high business impact.

3.2 Confirmed attack path only

Unauthenticated sender
-> sends specially crafted email to target mailbox
-> user opens message in on-premises OWA
-> certain interaction conditions are met
-> JavaScript executes in the user's browser context

That is the public, confirmed level of detail. The report intentionally does not expand this into exploit payload structure.

3.3 Important boundaries

This is not publicly described as direct Exchange server remote code execution. The public description places execution in the user's browser context. That still matters: browser-context code running on a trusted OWA origin can affect confidentiality and integrity of what that user can access in that browser session. However, defenders should avoid writing reports that claim server takeover, MFA bypass, session-token theft, or persistence unless evidence in their own environment supports those findings.

4. Lab architecture

4.1 Goal

Build a realistic but isolated lab to validate defensive controls, not exploitation.

4.2 Lab network

Use VirtualBox Internal Network named ExchangeLab-Internal. Do not use Bridged Adapter.

VM	Role	IP	Notes
LAB-DC01	Domain Controller, DNS	10.77.10.10	exchange-lab.test only
LAB-EX01	Exchange Server	10.77.10.20	OWA target for mitigation

VM	Role	IP	Notes
			validation
LAB-WIN01	Windows client	10.77.10.30	Browser, screenshots, functional tests
LAB-LOG01	Optional logging VM	10.77.10.40	Sysmon/WEF/ SIEM testing if desired

4.3 VirtualBox hard isolation checklist

Perform these settings on every lab VM before testing:

1. Settings -> Network -> Adapter 1 -> Attached to: Internal Network.
2. Name: ExchangeLab-Internal.
3. Disable Adapter 2, Adapter 3, and Adapter 4.
4. Settings -> General -> Advanced -> Shared Clipboard: Disabled.
5. Settings -> General -> Advanced -> Drag'n'Drop: Disabled.
6. Settings -> Shared Folders -> remove all shared folders.
7. Settings -> USB -> disable USB controller unless needed for installation media.
8. Take a snapshot named Clean-Isolated-Before-CVE42897-Lab.
9. In each VM, run:

```
ipconfig /all
route print
Test-NetConnection 8.8.8.8 -Port 53
Test-NetConnection officeclient.microsoft.com -Port 443
```

Expected isolated result during testing:

No default gateway, or no route to the internet.
External connectivity tests fail.
Only 10.77.10.0/24 lab hosts respond.

4.4 Controlled update/download window

Exchange needs official media, updates, and tools. Use this controlled process:

1. Before testing, temporarily attach the Exchange VM to NAT only for official Microsoft downloads.
2. Download only from Microsoft, Microsoft Learn, MSRC, and Microsoft GitHub repositories.
3. Save hashes and URLs in the evidence log.
4. Disconnect NAT and return Adapter 1 to Internal Network only.
5. Re-run the isolation checklist.
6. Take a snapshot named Official-Tools-Downloaded-Then-Isolated.

Do not conduct CVE-related testing while the lab has internet access.

5. Exchange and OWA baseline build

5.1 Supported-platform notes

For a current defensive lab, prefer Exchange Server SE on Windows Server 2022 or 2025 where licensing and media access permit. Microsoft lists Exchange SE as supporting Windows Server 2025, 2022, and 2019, and Exchange 2019 as supporting Windows Server 2025, 2022, and 2019. Exchange 2016 and 2019 are out of support except where an organisation has relevant arrangements. [S4]

5.2 Build order

1. Build LAB-DC01.
2. Configure static IP and DNS.
3. Promote LAB-DC01 to domain controller for exchange-lab.test.
4. Build LAB-EX01 and join it to the domain.
5. Install Exchange prerequisites from Microsoft documentation for your version.
6. Install Exchange using Microsoft-supported setup.
7. Build LAB-WIN01 and join it to the domain.
8. Create test users and mailboxes.
9. Confirm ordinary internal mail flow.
10. Confirm OWA access from LAB-WIN01.

5.3 Domain controller setup - configuration-changing commands

Run on LAB-DC01 from an elevated PowerShell session:

```
# CHANGE: install Active Directory Domain Services and DNS roles.
Install-WindowsFeature AD-Domain-Services,DNS -IncludeManagementTools

# CHANGE: create the test forest. Use a lab-only password.
Install-ADDSForest `
  -DomainName "exchange-lab.test" `
  -DomainNetbiosName "EXLAB" `
  -InstallDNS `
  -Force
```

Expected outcome: server restarts and becomes the domain controller for exchange-lab.test.

5.4 Test users

After Exchange is installed, create two normal lab mailboxes in Exchange Management Shell:

```
# CHANGE: create two fake mail-enabled users.
New-Mailbox -Name "Sender Test" `
  -Alias sender `
  -UserPrincipalName sender@exchange-lab.test `
  -Password (ConvertTo-SecureString 'Use-A-Lab-Only-Password-Here!' -AsPlainText
-Force) `
  -FirstName Sender `
  -LastName Test `
  -ResetPasswordOnNextLogon $false

New-Mailbox -Name "Recipient Test" `
  -Alias recipient `
  -UserPrincipalName recipient@exchange-lab.test `
  -Password (ConvertTo-SecureString 'Use-A-Lab-Only-Password-Here!' -AsPlainText
-Force) `
  -FirstName Recipient `
  -LastName Test `
  -ResetPasswordOnNextLogon $false
```

5.5 Baseline OWA smoke tests

From LAB-WIN01:

1. Browse to https://lab-ex01.exchange-lab.test/owa.
2. Sign in as sender@exchange-lab.test.
3. Send a plain-text message to recipient@exchange-lab.test.
4. Send a normal HTML message using formatting controls: bold text, bullet list, and a harmless hyperlink to https://example.test.
5. Sign out.
6. Sign in as recipient@exchange-lab.test.
7. Confirm both messages display normally.
8. Capture screenshots: login page, inbox, opened plain text message, opened normal HTML message.

Do not send malformed HTML, executable HTML, script tags, event handlers, or payloads designed to test the live CVE.

6. Baseline evidence collection

Create a folder on LAB-EX01:

```
# CHANGE: create local evidence folder.
New-Item -ItemType Directory -Path C:\CVE42897-Evidence -Force
```

6.1 Exchange build number

Microsoft documents Get-Command ExSetup.exe | ForEach-Object {$_.FileVersionInfo} as a way to confirm Exchange file version including security/hotfix update state. [S6]

```
Get-Command ExSetup.exe | ForEach-Object {$_.FileVersionInfo} |
  Out-File C:\CVE42897-Evidence\01-ExchangeBuild.txt
```

Expected output includes ProductVersion, FileVersion, and the path to ExSetup.exe.

6.2 Exchange server mitigation fields

Microsoft documents viewing applied and blocked mitigations with Get-ExchangeServer. [S3]

```
Get-ExchangeServer | Format-List
Name,Edition,AdminDisplayVersion,MitigationsEnabled,MitigationsApplied,MitigationsB
locked |
  Out-File C:\CVE42897-Evidence\02-ExchangeMitigationFields.txt
```

Expected outcome: MitigationsEnabled should be true unless intentionally disabled. MitigationsApplied should include the relevant M2/M2.1.x mitigation after automatic or manual application.

6.3 EM service status

```
Get-Service MSExchangeMitigation |
  Format-List Name,DisplayName,Status,StartType |
  Out-File C:\CVE42897-Evidence\03-MSExchangeMitigation-Service.txt
```

Expected outcome: service exists and is running, unless your lab is intentionally using manual EOMT validation only.

6.4 EM connectivity during controlled online phase

Microsoft documents the Test-MitigationServiceConnectivity.ps1 script in the Exchange V15\Scripts folder and states that a success result means the mitigation endpoint is accessible. [S3]

Run only during the controlled online phase:

```
cd "$env:ExchangeInstallPath\Scripts"
.\Test-MitigationServiceConnectivity.ps1 |
  Out-File C:\CVE42897-Evidence\04-MitigationServiceConnectivity.txt
```

Expected connected result:

Result: Success.
Message: The Mitigation Service endpoint is accessible from this computer.

6.5 OWA response headers

From LAB-WIN01, after signing in to OWA and loading the OWA shell:

1. Press F12 to open Developer Tools.
2. Open the Network tab.
3. Reload OWA.
4. Select the primary OWA HTML document.
5. Save or screenshot the Response Headers.
6. After mitigation, verify Content-Security-Policy contains script-src-attr 'none' for the relevant OWA HTML shell response.

Do not use an exploit email to validate this control. Header presence is the safe validation point.

7. Official mitigation validation

7.1 What Microsoft's mitigation does

Microsoft's CSS-Exchange mitigation definition for CVE-2026-42897 creates an outbound IIS URL Rewrite rule on the OWA virtual directory. Its stated purpose is to append a Content-Security-Policy header containing script-src-attr 'none' to HTML responses, blocking inline script event-handler attributes as an XSS mitigation. It limits the rule to specific OWA SPA HTML shell pages. [S5]

The same code checks for a matching outbound rule on IIS:\Sites\Default Web Site\owa that rewrites RESPONSE_Content_Security_Policy to include {R:1}; script-src-attr 'none', and checks for preconditions matching RESPONSE_CONTENT_TYPE of ^text/html and OWA request URI patterns. [S5]

7.2 Validate automatic EM application

Run in Exchange Management Shell:

```
Get-ExchangeServer | Format-List Name,MitigationsApplied,MitigationsBlocked
```

Expected result:

```
Name              : LAB-EX01
MitigationsApplied : {M2...}
MitigationsBlocked : {}
```

The exact sub-version may be shown as M2.1.x. If no M2/M2.1.x entry appears, do not assume protection.

7.3 Reapply automatic mitigations if EM is enabled

Microsoft documents restarting the EM service as a way to trigger a recheck; after restart, EM runs its check and applies any mitigations after about 10 minutes. [S3]

```
# CHANGE: restarts Exchange Emergency Mitigation service.
Restart-Service MSExchangeMitigation
```

Wait 10 minutes, then run:

```
Get-ExchangeServer -Identity LAB-EX01 | Format-List Name,*Mitigations*
```

7.4 Manual EOMT path for disconnected labs

Use the official Microsoft CSS-Exchange/EOMT package only. Do not use third-party scripts or copied blog code.

Process:

1. During the controlled online phase, download the latest CSS-Exchange/EOMT package from Microsoft GitHub.
2. Record the download URL, timestamp, and file hash.
3. Disconnect the lab from the internet.
4. Open elevated Exchange Management Shell.
5. Run the official EOMT invocation for CVE-2026-42897 as documented by the tool help in the downloaded package.
6. Capture all console output to C:\CVE42897-Evidence.

Example evidence wrapper:

```
Start-Transcript C:\CVE42897-Evidence\05-EOMT-Run.txt
# CHANGE: run the official Microsoft EOMT tool using its documented CVE-specific parameter.
# Example form, verify against the current tool help before running:
# .\EOMT.ps1 -CVE "CVE-2026-42897"
Stop-Transcript
```

7.5 Verify the outbound rewrite rule directly

Do not rely only on Exchange Health Checker for this CVE. A Microsoft CSS-Exchange issue explains that Health Checker was not enumerating IIS outbound URL Rewrite rules, meaning mitigations deployed as outbound rules, including EOMT OWA CSP - outbound, may be invisible in that report. [S7]

Run on LAB-EX01:

```
Import-Module WebAdministration

Get-WebConfiguration `
```

```
  -PSPath 'IIS:\Sites\Default Web Site\owa' `
  -Filter 'system.webServer/rewrite/outboundRules/rule' |
  Select-Object name,enabled,preCondition |
  Format-Table -AutoSize |
  Out-File C:\CVE42897-Evidence\06-OWA-OutboundRules.txt
```

Then capture the action and match details:

```
Get-WebConfiguration `
  -PSPath 'IIS:\Sites\Default Web Site\owa' `
  -Filter "system.webServer/rewrite/outboundRules/rule[@name='EOMT OWA CSP -
outbound']" |
  Format-List * |
  Out-File C:\CVE42897-Evidence\07-EOMT-OWA-CSP-Rule.txt
```

Expected behaviour:

- Rule exists.
- Rule is enabled.
- Rule targets RESPONSE_Content_Security_Policy.
- Rule action appends script-src-attr 'none'.
- Rule is limited to OWA HTML shell responses through preconditions.

7.6 Verify the browser-visible control

From LAB-WIN01:

```
# This may show certificate warnings in a lab with self-signed certificates.
# Use the browser Network tab as the primary method if PowerShell cannot validate TLS.
Invoke-WebRequest -Uri "https://lab-ex01.exchange-lab.test/owa/" -UseBasicParsing |
  Select-Object -ExpandProperty Headers
```

Expected result after mitigation: the relevant OWA HTML response includes a Content-Security-Policy value containing:

```
script-src-attr 'none'
```

This validates the defensive header. It does not prove the live exploit would or would not work, and it avoids triggering the real vulnerability.

8. Safe functional testing matrix

ID / test	Steps and expected result	Failure meaning and evidence	Remediation
FT-01 OWA login	Sign in as sender. Expected: login succeeds.	OWA auth broken. Capture screenshot and IIS/Exchange	Check IIS, certificate, and Exchange services.

ID / test	Steps and expected result	Failure meaning and evidence	Remediation
		logs.	
FT-02 Inbox display	Open inbox. Expected: inbox loads.	OWA shell issue. Capture screenshot and browser console.	Check CSP side effects and rewrite scope.
FT-03 Plain message	Read a normal text message. Expected: message opens normally.	Mail rendering issue. Capture screenshot.	Review OWA and application logs.
FT-04 Ordinary HTML	Read a bold/list/link message. Expected: normal formatting displays.	HTML rendering regression. Capture screenshot.	Review CSP/header scope.
FT-05 Compose	Create a new message. Expected: compose pane works.	OWA JavaScript regression. Capture screenshot and console.	Check console errors and rewrite scope.
FT-06 Send	Send to recipient. Expected: delivery succeeds.	Transport or mailbox issue. Capture message trace.	Check transport services and queues.
FT-07 Reply	Reply to message. Expected: reply sends.	Compose/reply broken. Capture screenshot.	Review OWA/app logs.
FT-08 Forward	Forward message. Expected: forward sends.	Compose/forward broken. Capture screenshot.	Review OWA/app logs.
FT-09 Calendar	Open calendar and create a test event. Expected: calendar works.	UI impact. Capture screenshot and console.	Document impact; test rollback only if critical.
FT-10 Search	Search inbox for test subject. Expected: results appear.	Search issue. Capture screenshot.	Check Exchange search service.
FT-11 Attachment	Attach harmless text file. Expected: upload and send succeeds.	Attachment/upload issue. Capture screenshot.	Check OWA virtual directory and logs.
FT-12 CSP header	Inspect OWA HTML response. Expected: CSP contains script-src-attr 'none'.	Mitigation not visible to browser. Capture DevTools export.	Re-run EM/EOMT and direct IIS checks.
FT-13 Logs	Check mitigation logs. Expected: no	EM issue. Capture Event IDs and	Fix OCS connectivity or use EOMT.

ID / test	Steps and expected result	Failure meaning and evidence	Remediation
	EM errors.	mitigation log file.	

No functional test requires executing JavaScript from an email in OWA.

9. Separate generic XSS learning exercise

This section teaches the vulnerability class in a toy app. It is not Exchange, not OWA, not email, not a CVE reproduction, and not a bypass. It runs only on LAB-WIN01 or another isolated lab VM.

9.1 Learning goal

Show the difference between:

1. Unsafe rendering of stored user-controlled HTML.
2. Safe HTML output encoding.
3. CSP with script-src-attr 'none' blocking inline event-handler attributes.

OWASP states that output encoding converts untrusted input into a safe form where it is displayed as data rather than executing as code. OWASP also describes CSP as a defence-in-depth layer that can make XSS harder to exploit, while not replacing secure coding. [S8][S9]

MDN documents that script-src-attr 'none' blocks inline event handlers. [S10]

9.2 Toy app code

Save as toy_xss_csp_lab.py on LAB-WIN01:

```
from http.server import ThreadingHTTPServer, BaseHTTPRequestHandler
from urllib.parse import parse_qs
from html import escape

HOST = "127.0.0.1"
PORT = 8088

notes = []
SAMPLE_NOTE = """<button
onclick=\"document.getElementById('status').textContent='LOCAL TOY DEMO: inline
handler ran'; return false;\">Local visual marker</button>"""

PAGE = """<!doctype html>
<html>
<head>
  <meta charset='utf-8'>
```

```
  <title>Toy XSS and CSP Lab</title>
  <style>
    body {{ font-family: Arial, sans-serif; max-width: 900px; margin: 40px auto; }}
    textarea {{ width: 100%; height: 90px; }}
    .note {{ border: 1px solid #bbb; padding: 10px; margin: 8px 0; }}
    code {{ background: #eee; padding: 2px 4px; }}
  </style>
</head>
<body>
  <h1>Toy XSS and CSP Lab</h1>
  <p>This is a local-only toy app. It does not use Exchange, OWA, email, cookies, tokens,
or network callbacks.</p>
  <p><strong>Status:</strong> <span id='status'>No inline handler has run.</span></p>
  <nav>
    <a href='/unsafe'>Unsafe rendering</a> |
    <a href='/safe'>Safe encoding</a> |
    <a href='/csp'>Unsafe rendering with CSP</a> |
    <a href='/reset'>Reset</a>
  </nav>
  <form method='post' action='/add'>
    <p><textarea name='note'>{sample}</textarea></p>
    <p><button type='submit'>Store note</button></p>
  </form>
  <h2>{mode}</h2>
  {notes_html}
</body>
</html>"""

class Handler(BaseHTTPRequestHandler):
    def render(self, mode):
        if mode == 'safe':
            rendered = ''.join(f"<div class='note'>{escape(n)}</div>" for n in notes)
        else:
            rendered = ''.join(f"<div class='note'>{n}</div>" for n in notes)

        html = PAGE.format(mode=mode, notes_html=rendered,
sample=escape(SAMPLE_NOTE))
        self.send_response(200)
        self.send_header('Content-Type', 'text/html; charset=utf-8')
        if mode == 'csp':
            self.send_header('Content-Security-Policy', "default-src 'self'; script-src 'self'; script-
src-attr 'none'")
        self.end_headers()
```

```
        self.wfile.write(html.encode('utf-8'))

    def do_GET(self):
        if self.path.startswith('/reset'):
            notes.clear()
            self.send_response(302)
            self.send_header('Location', '/unsafe')
            self.end_headers()
        elif self.path.startswith('/safe'):
            self.render('safe')
        elif self.path.startswith('/csp'):
            self.render('csp')
        else:
            self.render('unsafe')

    def do_POST(self):
        length = int(self.headers.get('Content-Length', '0'))
        body = self.rfile.read(length).decode('utf-8')
        data = parse_qs(body)
        notes.append(data.get('note', [''])[0])
        self.send_response(302)
        self.send_header('Location', '/unsafe')
        self.end_headers()

if __name__ == '__main__':
    print(f'Local toy lab: http://{HOST}:{PORT}/unsafe')
    ThreadingHTTPServer((HOST, PORT), Handler).serve_forever()
```

Run it:

```
python .\toy_xss_csp_lab.py
```

Open:

```
http://127.0.0.1:8088/unsafe
```

9.3 What to observe

1. Store the built-in sample note.
2. On /unsafe, click the local visual marker button. The status text changes, proving unsafe rendering can execute inline event-handler code in the toy app.
3. On /safe, the same note is displayed as text, not active HTML.
4. On /csp, the same unsafe rendering is present, but the browser should block the inline event handler because the response contains script-src-attr 'none'.
5. Open Developer Tools -> Console and observe the CSP violation message.

This teaches the general control Microsoft is applying, but it does not validate or reproduce CVE-2026-42897.

10. Detection and hunting plan

Because Microsoft has not published an exact payload or official IOC set for this CVE, detection should be behavioural and evidence-driven.

10.1 Preserve evidence first

1. Export or preserve the suspicious email without opening it in OWA.
2. Preserve message headers and transport metadata.
3. Preserve relevant IIS logs from Exchange Client Access/OWA paths.
4. Preserve mailbox audit logs.
5. Preserve endpoint browser and EDR telemetry where available.
6. Record exact user statement: OWA or Outlook desktop, preview or opened, clicked or not clicked, time, browser, and device.

10.2 Exchange and mailbox hunting

Run with appropriate date bounds around known exposure:

```
# Message tracking around suspected delivery.
Get-MessageTrackingLog -Start "2026-05-14 00:00" -End "2026-06-05 23:59" `
  -Recipients recipient@exchange-lab.test |
  Select-Object
Timestamp,EventId,Source,Sender,Recipients,MessageSubject,ClientIp,ServerIp
```

Mailbox rule review:

```
Get-InboxRule -Mailbox recipient@exchange-lab.test |
  Select-Object
Name,Enabled,Priority,ForwardTo,ForwardAsAttachmentTo,RedirectTo,DeleteMessage,
MoveToFolder |
  Format-List
```

Mailbox forwarding review:

```
Get-Mailbox recipient@exchange-lab.test |
  Format-List
DisplayName,ForwardingAddress,ForwardingSmtpAddress,DeliverToMailboxAndForwa
rd
```

Audit log review, adjusted to your enabled auditing configuration:

```
Search-MailboxAuditLog -Identity recipient@exchange-lab.test `
  -StartDate "05/14/2026" `
```

```
  -EndDate "06/05/2026" `
  -ShowDetails
```

10.3 IIS/OWA telemetry

Collect IIS logs from Exchange servers serving OWA:

```
Get-ChildItem C:\inetpub\logs\LogFiles -Recurse -Filter *.log |
  Where-Object LastWriteTime -ge (Get-Date).AddDays(-30) |
  Select-Object FullName,Length,LastWriteTime
```

Investigate:

- Unusual OWA user agents.
- Login or session anomalies for users who received suspicious messages.
- OWA access from unexpected source IPs.
- Timing correlation: suspicious message delivered -> user opened OWA -> unusual mailbox activity.
- Browser or EDR events showing unexpected script errors or network calls from the OWA tab.

Do not build a detector around guessed HTML strings or guessed event handlers. That risks false confidence and easy bypass.

11. Incident-response tabletop

Scenario: A user reports they opened a suspicious email in on-premises OWA before CVE-2026-42897 mitigation was confirmed.

11.1 First 30 minutes

1. Record user, device, time, browser, and exact interaction.
2. Ask the user to stop interacting with OWA on that device.
3. Preserve the email without opening it again in OWA.
4. Confirm whether the user used OWA, Outlook desktop, mobile Outlook, or another client.
5. Confirm whether M2 mitigation was applied at the time of opening.
6. Capture current mailbox rules and forwarding settings.
7. Capture recent OWA sign-ins/access logs for the user.
8. If evidence suggests active misuse, disable OWA access for the user temporarily or force sign-out/session revocation according to organisational process.

11.2 Containment

Precautionary containment may include:

- Password reset if credential compromise is plausible.

- Session revocation where supported.
- Review and removal of suspicious inbox rules.
- Temporary block of OWA access until mitigation is verified.
- Endpoint isolation if EDR or browser telemetry suggests additional compromise.

Do not state that MFA was bypassed or tokens were stolen unless your evidence supports that.

11.3 Scope assessment

1. Search for similar messages sent to other users.
2. Identify who opened similar messages in OWA.
3. Compare opening times to mailbox/audit activity.
4. Confirm which Exchange servers served OWA during the exposure period.
5. Confirm each server's mitigation status and timestamps.
6. Preserve evidence before cleanup.

11.4 Recovery and lessons learned

- Apply or verify M2 mitigation everywhere OWA is served.
- Disable or restrict internet-exposed OWA if mitigation cannot be verified.
- Patch to Microsoft's permanent security update when released.
- Update helpdesk scripts for suspicious OWA email reports.
- Add a recurring Exchange mitigation verification control.
- Review the strategic plan for reducing on-premises Exchange exposure.

12. Production change plan

12.1 Change title

Verify and apply Microsoft mitigation for CVE-2026-42897 on all on-premises Exchange OWA servers.

12.2 Risk statement

CVE-2026-42897 is known exploited and can lead to JavaScript execution in the browser context of users opening specially crafted emails in OWA under certain conditions. CISA required mitigation by 29 May 2026 for covered US federal agencies. Unverified mitigation leaves the organisation exposed to a high-impact mailbox and identity-adjacent risk. [S1]

12.3 Pre-change checks

- Confirm server inventory and OWA exposure.
- Confirm Exchange version and build using ExSetup.exe file version.
- Confirm EM service status.
- Confirm OCS connectivity or prepare EOMT.
- Export current IIS rewrite configuration.

- Snapshot or backup lab systems; for production, confirm supported backup and rollback plan.
- Notify helpdesk of possible OWA UI side effects.

12.4 Implementation

Preferred path:

1. Ensure EM service is enabled.
2. Confirm connectivity to OCS.
3. Restart MSExchangeMitigation if immediate recheck is required.
4. Wait 10 minutes.
5. Confirm M2/M2.1.x appears in mitigation state.
6. Verify outbound OWA rewrite rule directly.
7. Verify browser-visible CSP header.
8. Complete OWA functional test matrix.

Disconnected path:

1. Download official Microsoft EOMT during controlled online access.
2. Transfer through approved change channel.
3. Run EOMT in elevated Exchange Management Shell.
4. Verify the outbound rule and response header.
5. Complete functional tests.

12.5 Rollback

Rollback should only be considered if there is a critical service-impacting issue and after risk acceptance by business owner and IT/security lead.

For EM-managed mitigations, Microsoft documents blocking a mitigation with Set-ExchangeServer -Identity <ServerName> -MitigationsBlocked @("M2"), but blocking does not automatically remove it; IIS rule removal depends on mitigation type. [S3]

Rollback evidence must include:

- Business approval.
- Reason for rollback.
- Exact time.
- Rule removed or disabled.
- Compensating controls, such as temporarily disabling external OWA.
- Plan to restore mitigation.

13. Final report template

Use this template after running the lab or production verification.

Report title: CVE-2026-42897 Exchange OWA mitigation verification
Date:
Prepared by:
Environment:
Scope:

1. Sources reviewed
- Microsoft Security Response Center advisory:
- Microsoft Exchange Team guidance:
- Microsoft Learn EM service documentation:
- CSS-Exchange EOMT mitigation code:
- CISA KEV/NVD:

2. Confirmed facts
- CVE description:
- Affected servers:
- Current Microsoft mitigation:
- Patch/build status:

3. Unknowns and assumptions
- Exact payload unknown:
- Interaction condition unknown:
- No official IOCs identified:

4. Lab/production architecture
- Servers:
- OWA exposure:
- Client/browser used for testing:

5. Evidence collected
- Exchange build:
- EM service status:
- MitigationsApplied:
- IIS outbound rewrite rule:
- CSP response header:
- Functional test screenshots:
- Logs reviewed:

6. Results
- Mitigation status:
- OWA functionality:
- Gaps found:
- Remediation actions:

7. Risk decision
- Accepted risk:
- Required follow-up:
- Owner:
- Due date:

8. Patch-monitoring plan
- Monitor MSRC:
- Monitor Exchange Team blog:
- Monitor Exchange build list:
- Retest after security update:

14. One-page IT manager checklist

Immediate

- ☐ Identify every on-premises Exchange server serving OWA.
- ☐ Confirm Exchange Online-only tenants are not using on-premises OWA for mailboxes.
- ☐ Record Exchange build using Get-Command ExSetup.exe | ForEach-Object {$_.FileVersionInfo}.
- ☐ Confirm EM service exists and is running.
- ☐ Confirm MitigationsEnabled is true at server/organisation level.
- ☐ Confirm EM connectivity to OCS or prepare official EOMT.
- ☐ Verify M2/M2.1.x mitigation state.
- ☐ Verify the OWA outbound rewrite rule directly in IIS, not only via Health Checker.
- ☐ Verify OWA response header contains script-src-attr 'none'.
- ☐ Run OWA functional smoke tests.

If mitigation cannot be verified

- ☐ Escalate as active high risk.
- ☐ Restrict or disable external OWA until mitigation is verified.
- ☐ Use official EOMT if EM cannot reach OCS.
- ☐ Open a vendor/support case if mitigation cannot be applied.

Investigation

- ☐ Preserve suspicious emails without reopening in OWA.
- ☐ Review affected users' mailbox rules and forwarding.
- ☐ Review OWA/IIS logs and mailbox audit logs around suspicious email opening times.
- ☐ Use behavioural hunting; do not rely on guessed payload strings.

Longer term

- ☐ Monitor MSRC and Exchange Team for permanent security update.

- ☐ Patch when Microsoft releases a permanent fix.
- ☐ Re-test CSP/mitigation state after patching.
- ☐ Remove obsolete mitigation only when Microsoft guidance says it is safe.
- ☐ Review whether internet-exposed on-premises OWA is still necessary.

References

[S1] NVD, CVE-2026-42897 Detail. https://nvd.nist.gov/vuln/detail/CVE-2026-42897

[S2] Microsoft Exchange Team, "Addressing Exchange Server May 2026 vulnerability CVE-2026-42897", Microsoft Tech Community, 14 May 2026. https://techcommunity.microsoft.com/blog/exchange/addressing-exchange-server-may-2026-vulnerability-cve-2026-42897/4518498

[S3] Microsoft Learn, "Exchange Emergency Mitigation Service". https://learn.microsoft.com/en-us/exchange/plan-and-deploy/post-installation-tasks/security-best-practices/exchange-emergency-mitigation-service

[S4] Microsoft Learn, "Exchange Server supportability matrix". https://learn.microsoft.com/en-us/exchange/plan-and-deploy/supportability-matrix

[S5] Microsoft CSS-Exchange GitHub, CVE-2026-42897.ps1 mitigation definition. https://github.com/microsoft/CSS-Exchange/blob/main/Security/src/EOMT/Mitigations/CVE-2026-42897.ps1

[S6] Microsoft Learn, "Exchange Server build numbers and release dates". https://learn.microsoft.com/en-us/exchange/new-features/build-numbers-and-release-dates

[S7] Microsoft CSS-Exchange GitHub issue #2539, "Health Checker does not detect IIS outbound rewrite rules". https://github.com/microsoft/CSS-Exchange/issues/2539

[S8] OWASP Cheat Sheet Series, "Cross Site Scripting Prevention". https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

[S9] OWASP Cheat Sheet Series, "Content Security Policy". https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

[S10] MDN Web Docs, "Content-Security-Policy: script-src-attr directive". https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Security-Policy/script-src-attr

Appendix A - Research limitations

- The MSRC page requires JavaScript in some retrieval contexts, so this report cross-checks MSRC-derived information through NVD/CNA data, the Microsoft Exchange Team post, Microsoft Learn, and Microsoft CSS-Exchange code.
- The exact CVE exploit structure is intentionally excluded.

- No production claims should be made until commands are run in the actual environment.
- Third-party articles were used only to check whether a later permanent patch had been reported; they were not used as the basis for exploit steps.